

# TAKSHASHILA UNIVERSITY

## Tamil Nadu State Private University

( Established under Tamil Nadu State Private Universities Act 2019

& Recognized by UGC u/s 2(f) of the UGC Act,1956 )

Ongur, Tindivanam Taluk, Villupuram District, Tamil Nadu – 604305.



**TAKSHASHILA  
UNIVERSITY**

**FACULTY OF SCIENCES**

## SCHOOL OF COMPUTER SCIENCE

### STUDENT RECORD

**REGISTER NUMBER** : \_\_\_\_\_

**NAME OF THE STUDENT** : \_\_\_\_\_

**PROGRAM NAME** : \_\_\_\_\_

**YEAR / SEMESTER** : \_\_\_\_\_

**COURSE CODE** : \_\_\_\_\_

**COURSE NAME** : \_\_\_\_\_

# TAKSHASHILA UNIVERSITY

## Tamil Nadu State Private University

( Established under Tamil Nadu State Private Universities Act 2019  
& Recognized by UGC u/s 2(f) of the UGC Act,1956 )  
Ongur, Tindivanam Taluk, Villupuram District, Tamil Nadu – 604305.

### FACULTY OF SCIENCES

### School of Computer Science

## Bonafide Certificate

|                        |  |
|------------------------|--|
| <b>Register Number</b> |  |
|------------------------|--|

*This is to certify that this is a Bonafide Record of practical work done by*

*Mr./Ms. .... of .....*

*..... Semester in the .....*

*Laboratory during the academic year .....*

*Signature of Subject Staff*

*Signature of School Incharge*

---

Submitted the University Practical Examination held on .....

Signature Internal Examiner

Signature External Examiner

## INDEX PAGE

| S.NO | DATE | PROGRAM                                   | SIGNATURE |
|------|------|-------------------------------------------|-----------|
| 1    |      | Insert Multiple Documents                 |           |
| 2    |      | CRUD Operations in MongoDB                |           |
| 3    |      | Creating Array Fields in MongoDB          |           |
| 4    |      | Query Operators in MongoDB                |           |
| 5    |      | Aggregation Function in MongoDB           |           |
| 6    |      | Sorting and Limiting Documents in MongoDB |           |
| 7    |      | Creating and Viewing Indexes in MongoDB   |           |
| 8    |      | Projection in MongoDB                     |           |

|           |                           |
|-----------|---------------------------|
| EX NO : 1 | Insert Multiple Documents |
|           |                           |

**AIM :**

To insert multiple documents into a MongoDB collection using the insertMany() method.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Show available databases using show dbs.

Step 3: Create/select database using use CollegeDB.

Step 4: Insert sample documents using insertMany().

Step 5: Create an index on rollno using createIndex().

Step 6: View all indexes using getIndexes().

### Program :

```
db.students.insertMany([
  { name: "ragul", age: 21, course: "BCA" },
  { name: "Mugesh", age: 22, course: "BSc CS" },
  { name: "Anitha", age: 20, course: "BTech" }
]);
db.students.find();
```

### OUTPUT :

```
ragul> db.students.insertMany([
...   { name: "Ragul", age: 20, department: "CSE" },
...   { name: "Mugesh", age: 21, department: "IT" },
...   { name: "Anitha", age: 19, department: "ECE" }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('691ff43b88c0630cbd63b112'),
    '1': ObjectId('691ff43b88c0630cbd63b113'),
    '2': ObjectId('691ff43b88c0630cbd63b114')
  }
}
```

**Result:**

The program successfully inserted multiple documents into the *students* collection using `insertMany()` and displayed them correctly with the `find()` command.

|           |                            |
|-----------|----------------------------|
| EX NO : 2 | CRUD Operations in MongoDB |
|           |                            |

**AIM :**

To perform Insert, Read, Update, and Delete (CRUD) operations in MongoDB using basic commands.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Show all existing databases using show dbs.

Step 3: Create / switch to a database using use EmployeeDB.

Step 4: Insert a document into collection using insertOne().

Step 5: View all documents using find().

Step 6: Update a field using updateOne().

Step 7: Delete a document using deleteOne().

Step 8: Verify results using find() again

## PROGRAM :

```
show dbs;

use EmployeeDB;

db.employees.insertOne({
  name: "Arun",
  salary: 25000,
  place: "Chennai"
});

db.employees.find();

db.employees.updateOne(
  { name: "Arun" },
  { $set: { salary: 30000 } }
);

db.employees.find();

db.employees.deleteOne({ name: "Arun" });

db.employees.find();
```

## OUTPUT :

```
[
  {
    _id: ObjectId('691ffa3f88c0630cbd63b116'),
    name: 'Arun',
    salary: 25000,
    place: 'Chennai'
  }
]
```

```
[
  {
    _id: ObjectId('691ffa3f88c0630cbd63b116'),
    name: 'Arun',
    salary: 30000,
    place: 'Chennai'
  }
]
```



**RESULT:**

CRUD operations were successfully performed using MongoDB. Insert, Read, Update, and Delete commands executed correctly.

|           |                                  |
|-----------|----------------------------------|
| EX NO : 3 | Creating Array Fields in MongoDB |
|           |                                  |

**AIM :**

To create a collection with **array-type fields** and retrieve documents containing array values in MongoDB.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Display all databases using show dbs.

Step 3: Create or switch to a database using use ProductDB.

Step 4: Create a collection and insert a document with an array field.

Step 5: Display all collections using show collections.

Step 6: Use find() to display inserted documents.

## PROGRAM:

```
db.items.insertOne({  
  product: "Laptop",  
  brands: ["Dell", "HP", "Lenovo", "Asus"]  
});
```

```
db.items.insertOne({  
  product: "Mobile",  
  brands: ["Samsung", "Apple", "Vivo", "Realme"]  
});
```

```
db.items.find()
```

## OUTPUT :

```
[  
  {  
    _id: ObjectId('691ffd4488c0630cbd63b117'),  
    product: 'Laptop',  
    brands: [ 'Dell', 'HP', 'Lenovo', 'Asus' ]  
  },  
  {  
    _id: ObjectId('691ffd4488c0630cbd63b118'),  
    product: 'Mobile',  
    brands: [ 'Samsung', 'Apple', 'Vivo', 'Realme' ]  
  }  
]
```

**RESULT:**

A collection with array fields was successfully created. Documents containing arrays were inserted and retrieved using MongoDB commands.

|           |                            |
|-----------|----------------------------|
| EX NO : 4 | Query Operators in MongoDB |
|           |                            |

**AIM :**

To use MongoDB query operators such as \$gt, \$lt, \$gte, \$lte to filter documents based on conditions,

**PROCEDURE:**

Step 1: Open MongoDB shell.

Step 2: Use show dbs to list databases.

Step 3: Create/Select a database using use MarksDB.

Step 4: Insert marks of students using insertMany().

Step 5: Write queries using \$gt, \$gte, \$lte operators.

Step 6: Display the filtered results using find().

## PROGRAM :

```
use MarksDB;
```

```
db.scores.insertMany([
  { name: "Ragul", mark: 85 },
  { name: "Ravi", mark: 45 },
  { name: "Sita", mark: 95 },
  { name: "Kumar", mark: 67 },
  { name: "Anu", mark: 30 }
]);
```

```
db.scores.find({ mark: { $gt: 50 } });
```

```
db.scores.find({ mark: { $lte: 60 } });
```

```
db.scores.find({ mark: { $gte: 60, $lte: 90 } });
```

## OUTPUT :

```
MarksDB> db.scores.find({ mark: { $gt: 50 } });
[
  {
    _id: ObjectId('6920022388c0630cbd63b119'),
    name: 'Ragul',
    mark: 85
  },
  {
    _id: ObjectId('6920022388c0630cbd63b11b'), name: 'Sita', mark: 95 },
  {
    _id: ObjectId('6920022388c0630cbd63b11c'),
    name: 'Kumar',
    mark: 67
  }
]
```

```
MarksDB> db.scores.find({ mark: { $lte: 60 } });
[
  { _id: ObjectId('6920022388c0630cbd63b11a'), name: 'Ravi', mark: 45 },
  { _id: ObjectId('6920022388c0630cbd63b11d'), name: 'Anu', mark: 30 }
]
```

```
MarksDB> db.scores.find({ mark: { $gte: 60, $lte: 90 } });
[
  {
    _id: ObjectId('6920022388c0630cbd63b119'),
    name: 'Ragul',
    mark: 85
  },
  {
    _id: ObjectId('6920022388c0630cbd63b11c'),
    name: 'Kumar',
    mark: 67
  }
]
```

**RESULT:**

Query operators \$gt, \$lte, \$gte, and \$lt were successfully used to filter documents based on mark conditions.

|           |                                 |
|-----------|---------------------------------|
| EX NO : 5 | Aggregation Function in MongoDB |
|           |                                 |

AIM :

To perform aggregation in MongoDB using the \$group and \$sum operators to calculate total sales for each product.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Use show dbs to list all databases.

Step 3: Create/select database using use SalesDB.

Step 4: Insert sample sales data using insertMany().

Step 5: Use aggregation pipeline with \$group and \$sum.

Step 6: Display total sales for each product.



## PROGRAM ;

```
use SalesDB;
```

```
db.sales.insertMany([  
  { product: "TV", price: 20000 },  
  { product: "TV", price: 25000 },  
  { product: "Fridge", price: 30000 },  
  { product: "Washing Machine", price: 18000 },  
  { product: "Fridge", price: 28000 }  
]);
```

```
db.sales.aggregate([  
  {  
    $group: {  
      _id: "$product",  
      totalRevenue: { $sum: "$price" }  
    }  
  }  
]);
```

OUTPUT :

```
[  
  { _id: 'Fridge', totalRevenue: 58000 },  
  { _id: 'TV', totalRevenue: 45000 },  
  { _id: 'Washing Machine', totalRevenue: 18000 }  
]
```

**RESULT:**

Aggregation was successfully performed using \$group and \$sum, and total revenue for each product was calculated.

|           |                                           |
|-----------|-------------------------------------------|
| EX NO : 6 | Sorting and Limiting Documents in MongoDB |
|           |                                           |

**AIM:**

To perform sorting of documents in ascending and descending order and to limit the number of results using the sort() and limit() methods.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: View available databases using show dbs.

Step 3: Create/Select database using use CollegeDB.

Step 4: Insert student details using insertMany().

Step 5: Apply sorting in ascending & descending order.

Step 6: Limit output to first 2 documents.

## PROGRAM ;

use CollegeDB;

```
db.students.insertMany([
  { name: "Ragul", age: 20 },
  { name: "Kavin", age: 22 },
  { name: "Priya", age: 19 },
  { name: "Suresh", age: 23 }
]);
```

```
db.students.find().sort({ age: 1 });
db.students.find().sort({ age: -1 });
db.students.find().limit(2);
```

## OUTPUT :

```
CollegeDB> db.students.find().sort({ age: 1 });
[
  { _id: ObjectId('692010ee88c0630cbd63b125'), name: 'Priya', age: 19 },
  { _id: ObjectId('692010ee88c0630cbd63b123'), name: 'Ragul', age: 20 },
  { _id: ObjectId('692010ee88c0630cbd63b124'), name: 'Kavin', age: 22 },
  {
    _id: ObjectId('692010ee88c0630cbd63b126'),
    name: 'Suresh',
    age: 23
  }
]
```

```
CollegeDB> db.students.find().sort({ age: -1 });
[
  {
    _id: ObjectId('692010ee88c0630cbd63b126'),
    name: 'Suresh',
    age: 23
  },
  { _id: ObjectId('692010ee88c0630cbd63b124'), name: 'Kavin', age: 22 },
  { _id: ObjectId('692010ee88c0630cbd63b123'), name: 'Ragul', age: 20 },
  { _id: ObjectId('692010ee88c0630cbd63b125'), name: 'Priya', age: 19 }
]
```

```
CollegeDB> db.students.find().limit(2);
[
  { _id: ObjectId('692010ee88c0630cbd63b123'), name: 'Ragul', age: 20 },
  { _id: ObjectId('692010ee88c0630cbd63b124'), name: 'Kavin', age: 22 }
]
```

**RESULT:**

Sorting and limiting operations were successfully performed using `sort()` and `limit()` in MongoDB.

|           |                                         |
|-----------|-----------------------------------------|
| EX NO : 7 | Creating and Viewing Indexes in MongoDB |
|           |                                         |

**AIM :**

To create an index on a field using `createIndex()` and verify it using `getIndexes()`.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Show available databases using `show dbs`.

Step 3: Create/select database using `use CollegeDB`.

Step 4: Insert sample documents using `insertMany()`.

Step 5: Create an index on rollno using `createIndex()`.

Step 6: View all indexes using `getIndexes()`.

## PROGRAM :

```
db.students.insertMany([
  { name: "Ragul", rollno: 101, department: "CSE" },
  { name: "Mugesh", rollno: 102, department: "IT" },
  { name: "Priya", rollno: 103, department: "ECE" }
]);

db.students.createIndex({ rollno: 1 });

db.students.getIndexes();

db.students.find({ rollno: 102 }).explain("executionStats");
```

## OUTPUT :

```
}
ragul> db.students.createIndex({ rollno: 1 });
rollno_1
ragul> db.students.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollno: 1 }, name: 'rollno_1' }
]
ragul> db.students.find({ rollno: 102 }).explain("executionStats");
{
```

**RESULT:**

An index was successfully created on the **rollno** field, and queries used **IXSCAN**, showing improved performance.



|           |                       |
|-----------|-----------------------|
| EX NO : 8 | Projection in MongoDB |
|           |                       |

**AIM :**

To display only selected fields from documents using projection in MongoDB.

**PROCEDURE :**

Step 1: Open MongoDB shell.

Step 2: Show all databases using show dbs.

Step 3: Create/select database using use StudentDB.

Step 4: Insert sample documents using insertMany().

Step 5: Use projection to display only name and department.

Step 6: Use projection to hide \_id field.

## Program :

```
use StudentDB;
```

```
db.students.insertMany([
  { name: "Ragul", age: 20, department: "CSE", city: "Chennai" },
  { name: "Priya", age: 21, department: "IT", city: "Madurai" },
  { name: "Suresh", age: 22, department: "ECE", city: "Coimbatore" }
]);
```

```
db.students.find({}, { name: 1, department: 1 });
```

```
db.students.find({}, { _id: 0, name: 1, department: 1 });
```

## Output :

```
StudentDB> db.students.find({}, { name: 1, department: 1 });
[
  {
    _id: ObjectId('6920246a88c0630cbd63b12a'),
    name: 'Ragul',
    department: 'CSE'
  },
  {
    _id: ObjectId('6920246a88c0630cbd63b12b'),
    name: 'Priya',
    department: 'IT'
  },
  {
    _id: ObjectId('6920246a88c0630cbd63b12c'),
    name: 'Suresh',
    department: 'ECE'
  }
]
```

```
StudentDB> db.students.find({}, { _id: 0, name: 1, department: 1 });
[
  { name: 'Ragul', department: 'CSE' },
  { name: 'Priya', department: 'IT' },
  { name: 'Suresh', department: 'ECE' }
]
```

**RESULT:**

Projection was successfully applied using MongoDB's `find()` method, and only selected fields were displayed.

